

객체지향 프로그래밍

- 1차 과제 -

이름	김금영
학번	2022041052
학과	소프트웨어학부
담당교수	안광모
강의실	S4-1 205



객체지향 프로그래밍

실습 10-3 : List 클래스를 상속받아 큐처럼 작동하는 Queue 클래스 구현

소프트웨어학부

2022041052

김금영

1. 코드 구성 및 설명

1.1 전체 코드 구성

main.cpp

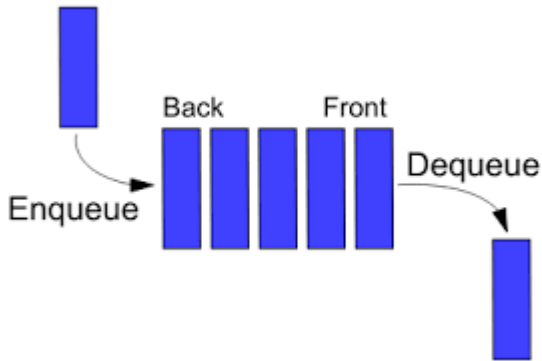
```
1: #include "Queue.h"
2: #include <iostream>
3: using namespace std;
4:
5: int main() {
6:     Queue queue;
7:     int input;
8:     int size;
9:
10:    cout << "큐에 입력할 사이즈를 입력해주세요 : ";
11:    cin >> size;
12:    cout << endl;
13:
14:    int queue_status[size];
15:
16:    for (int i = 1; i < size + 1; i++) {
17:        cout << i << "번째 큐 입력 값 : ";
18:        cin >> input;
19:        queue.enqueue(input);
20:        queue_status[i - 1] = input;
21:    }
22:    cout << endl;
23:
24:    cout << "현재 queue의 상황" << endl;
25:    cout << endl;
```

```
26:
27:    cout << " <-- dequeue " << " | ";
28:    for (int k = 0; k < size; k++) {
29:        cout << " " << queue_status[k] << " | ";
30:    }
31:    cout << " <-- enqueue  " << endl;
32:    cout << endl;
33:
34:    cout << "몇 번째 까지 출력 하실 건가요? : ";
35:    cin >> size;
36:    int count = 0;
37:
38:    for (int i = 0; i < size; i++) {
39:        cout << (i + 1) << "번째 dequeue : ";
40:        cout << queue.dequeue() << " " << endl;
41:        count++;
42:    }
43:
44:    cout << endl;
45:
46:    cout << "나머지 큐 전체 출력" << endl;
47:    count++;
48:
49:    while (!queue.empty()) {
50:        cout << count << "번째 dequeue" <<
queue.dequeue() << " " << endl;
51:        count++;
52:    }
53:
```

```
54: return 0;
55: }
```

main.cpp

main.cpp는 사용자에게 큐의 size를 입력받아.
size 만큼 enqueue()를 실행합니다.



실제 큐의 구조를 쉽게 알기 위해서
현재 큐의 상태를 출력하고,

현재 queue의 상황

```
<-- dequeue | 1 | 2 | 34 | 5 | 6 | 77 | <-- enqueue
```

사용자에게 dequeue() 할 요소를 입력받은 뒤
해당 큐를 dequeue 합니다.

마지막으로 큐의 모든 요소를 출력 한 뒤 종료합니다.

List.cpp

```
1: #include "List.h"
2: #include <iostream>
3: using namespace std;
4:
5: List::List()
6: : head(nullptr), tail(nullptr), size(0) {}
7:
8: List::~List() {
9: while (head) {
10:     ListNode *tmp = head->next;
11:     delete head;
```

```
12:     head = tmp;
13: }
14: }
15:
16: void List::pushFront(int value) {
17:     ListNode *node = new ListNode(value);
18:     if (!head)
19:         head = tail = node;
20:     else {
21:         head->prev = node;
22:         node->next = head;
23:         head = node;
24:     }
25:     size++;
26: }
27:
28: void List::pushBack(int value) {
29:     ListNode *node = new ListNode(value);
30:     if (!head)
31:         head = tail = node;
32:     else {
33:         tail->next = node;
34:         node->prev = tail;
35:         tail = node;
36:     }
37:     size++;
38: }
39:
40: int List::popFront() {
41:     if (!head)
42:         throw "Empty List";
43:
44:     int value = head->value;
45:     ListNode *del = head;
46:     head = head->next;
47:     if (head)
48:         head->prev = nullptr;
```

```

49:  else
50:      tail = nullptr;
51:  delete del;
52:  size--;
53:  return value;
54: }
55:
56: int List::popBack() {
57:     if (!tail)
58:         throw "Empty List";
59:
60:     int value = tail->value;
61:     ListNode *del = tail;
62:     tail = tail->prev;
63:     if (tail)
64:         tail->next = nullptr;
65:     else
66:         head = nullptr;
67:     delete del;
68:     size--;
69:     return value;
70: }
71:
72: int List::length() const {
73:     return size;
74: }

```

List.cpp

링크드리스트를 이용하여 List 클래스를 구현했습니다

연결리스트를 이용하여 양방향 탐색이 가능하고 앞/뒤 삽입 및 삭제 연산을 수행합니다.

- pushFront(int value)

맨 앞 부분에 노드를 추가 하는 함수입니다. 기존 노드 head에 새 노드를 연결하고 head포인터를 업데이트 한 후 size를 증가시킵니다.

- pushBack(int value)

맨뒤 부분에 노드를 추가하는 함수입니다. 만약 리스트가 비어 있다면 head와 tail이 해당 노드를 가르키도록 설정합니다. 만약 비어있지 않다면 기존의 tail노드 뒤에 노드를 연결한 후 tail 포인터를 새 노드로 갱신합니다.

- popFront()

맨 앞 부분의 노드를 삭제하고 해당노드를 출력하는 함수 입니다. 만약 리스트가 비어있지 않다면, head가 가르키는 노드의 값을 저장 후 head를 다음 노드로 이동시키고 기존의 노드를 해제 합니다.

-popBack()

마지막 노드를 삭제하고 해당 값을 출력하는 함수입니다. tail 노드의 값을 저장 후 tail을 이전으로 이동 시킨 뒤 마지막 노드를 삭제합니다.

List.h

```

1: #ifndef List_H
2: #define List_H
3:
4: class ListNode;
5:
6: class List {
7:     ListNode *head;
8:     ListNode *tail;
9:     int size;
10:
11: public:
12:     List();
13:     ~List();
14:
15:     void pushFront(int);
16:     void pushBack(int);

```

```

17:  int popFront();
18:  int popBack();
19:  int length() const;
20: };
21:
22: class ListNode {
23:     int value;
24:     ListNode *next;
25:     ListNode *prev;
26:
27: public:
28:     friend class List;
29:     ListNode(int value = 0)
30:         : value(value), next(nullptr), prev(nullptr) {}
31: };
32:
33: #endif

```

List.h

연결 리스트 구조 정의 헤더파일입니다. 각 포인터들은 next, prev 포인터를 가지고, 큐 형식과 마찬가지로 양방향으로 푸시와 팝을 하기위해 함수들을 정의해두었습니다.

Queue.cpp

```

1: #include "queue.h"
2:
3: Queue::Queue() : List() {}
4:
5: Queue::~Queue() {}
6:
7: void Queue::enqueue(int value) {
8:     pushBack(value);
9: }
10:
11: int Queue::dequeue() {
12:     return popFront();
13: }
14:
15: bool Queue::empty() const {
16:     return (length() == 0);
17: }

```

Queue.cpp

리스트 클래스 기반으로 선입선출의 큐를 직접 실행해보는 코드입니다.

- enqueue(int value)

큐의 뒤쪽에서 데이터를 추가하는 함수입니다. 리스트 클래스에서 pushBack함수를 호출 후 새로운 노드를 리스트 끝에 추가합니다.

- dequeue()

큐의 앞 부분에서 데이터를 출력하는 함수입니다. 리스트의 맨 앞 노드를 제거하고 popFront() 함수를 호출하여 해당 값을 출력합니다. 큐의 선입선출 구조를 구현하기 위해서 항상 데이터는 앞에서 출력하도록 구현하였습니다.

- empty()

해당 함수는 큐가 비어있는지 확인하는 함수입니다. 노드의 개수를 확인하여 0이라면 해당 큐가 비워있음을 확인할 수 있도록 구현하였습니다.

Queue.h

```
1: #ifndef QUEUE_H
2: #define QUEUE_H
3:
4: #include "List.h"
5:
6: class Queue : private List {
7: public:
8:     Queue();
9:     ~Queue();
10:
11:     void enqueue(int value); // 큐의 뒤에 값을 추가
12:     int dequeue();          // 큐의 앞에서 값을 제거하고
반환
13:
14:     bool empty() const;    // 큐가 비었는지 확인
15: };
16:
17: #endif
```

Queue.h

queue.cpp의 클래스를 정의하는 헤더파일 입니다.

enqueue, dequeue, empty함수를 통해 큐의 선입선출 동작을 정의합니다.

2 실행 결과

2.1 분할 컴파일

```
PS C:\WVS code\Programming_study\객체지향프로그래밍\과제> g++ -c List.cpp
PS C:\WVS code\Programming_study\객체지향프로그래밍\과제> g++ -c Queue.cpp
PS C:\WVS code\Programming_study\객체지향프로그래밍\과제> g++ -c main.cpp
PS C:\WVS code\Programming_study\객체지향프로그래밍\과제> ls

디렉터리: C:\WVS code\Programming_study\객체지향프로그래밍\과제

Mode                LastWriteTime         Length Name
----                -
-a----          2025-06-01 오후 9:07        194677 10-2.pdf
-a----          2025-06-03 오후 11:05         1262 List.cpp
-a----          2025-06-03 오후 10:42          496 List.h
-a----          2025-06-03 오후 11:36        3901 List.o
-a----          2025-06-03 오후 11:35         688 main.cpp
-a----          2025-06-03 오후 11:37        3261 main.o
-a----          2025-06-03 오후 11:00          256 Queue.cpp
-a----          2025-06-03 오후 10:53          218 Queue.h
-a----          2025-06-03 오후 11:36         1532 Queue.o
-a----          2025-06-01 오후 9:42         4760 과제.txt

PS C:\WVS code\Programming_study\객체지향프로그래밍\과제>
```

2.2 파일 링크

```
PS C:\WVS code\Programming_study\객체지향프로그래밍\과제> g++ List.o Queue.o m
ain.o -o queue_simulation
PS C:\WVS code\Programming_study\객체지향프로그래밍\과제> ls

디렉터리: C:\WVS code\Programming_study\객체지향프로그래밍\과제

Mode                LastWriteTime         Length Name
----                -
-a----          2025-06-01 오후 9:07        194677 10-2.pdf
-a----          2025-06-03 오후 11:05         1262 List.cpp
-a----          2025-06-03 오후 10:42          496 List.h
-a----          2025-06-03 오후 11:36        3901 List.o
-a----          2025-06-03 오후 11:35         688 main.cpp
-a----          2025-06-03 오후 11:37        3261 main.o
-a----          2025-06-03 오후 11:00          256 Queue.cpp
-a----          2025-06-03 오후 10:53          218 Queue.h
-a----          2025-06-03 오후 11:36         1532 Queue.o
-a----          2025-06-03 오후 11:37        61906 queue_simulation.exe
-a----          2025-06-01 오후 9:42         4760 과제.txt
```

2.3 실행결과

```
PS C:\WVS code\Programming_study\객체지향프로그래밍\과제> g++ List.o Queue.o
ain.o -o queue_simulation
PS C:\WVS code\Programming_study\객체지향프로그래밍\과제> ./queue_simulation
큐에 입력할 사이즈를 입력해주세요 : 6

1번째 큐 입력 값 : 1
2번째 큐 입력 값 : 2
3번째 큐 입력 값 : 34
4번째 큐 입력 값 : 5
5번째 큐 입력 값 : 6
6번째 큐 입력 값 : 77

현재 queue의 상황

<-- dequeue | 1 | 2 | 34 | 5 | 6 | 77 | <-- enqueue

몇 번째 까지 출력 하실 건가요? : 3
1번째 dequeue : 1
2번째 dequeue : 2
3번째 dequeue : 34

나머지 큐 전체 출력
4번째 dequeue5
5번째 dequeue6
6번째 dequeue77
```

과제 회고

해당 실습문제를 수행하면서 상속을 이용하여 선입선출 자료구조인 큐를 직접 구현해 보았습니다. 상속을 이용하여서 불필요한 코드 중복을 줄이고 간단하게 구현할 수 있었고, 또한 해당 코드들을 분할 컴파일을 진행해보고 직접 링크 및 실행을 해보면서 분할컴파일의 장점 (코드 모듈관리)를 알 수 있었습니다.